

Finding connected components in a graph

Database Management Project

Benjamin Cerf
Charles De Cian
Thanh Liêm Nguyen

MAP543 - Database Management
Dario Colazzo

Introduction

The aim of this project is to implement a MapReduce algorithm for finding connected components in a graph, and then to compare the performance of this algorithm when coded in Python or Scala, and when using RDDs or Dataframes.

A graph is a fundamental data structure used to represent relationships between entities. It is generally defined as a set of nodes V and a set of edges E . Each edge in the graph connects two nodes and can be directed or undirected, depending on whether the relationship between the nodes is asymmetrical or symmetrical, respectively. This notion of a graph is very important today, as it is applied in a wide variety of situations, when trying to study the relationships between certain entities in databases. For example, when observing social network databases.

The challenge of this project is to identify groups of connected components within a graph. A connected component is a sub-graph in which there is a path between each pair of nodes. Identifying a connected component therefore makes it possible to identify, for example, a community within a social network, and thus to understand the organization of complex systems.

The CCF (Connected Component Finder) algorithm is an algorithm described in a research article published in 2014 (see References). It is an algorithm for finding Connected Components. It is a MapReduce algorithm, which has outperformed other Connected Component detection algorithms in terms of speed and scalability. In particular, it demonstrated its ability to process a graph of 6B nodes and 92B edges on a hadoop cluster of 80 nodes, which was a record at the time the research paper was published.

The aim of this project is to implement this algorithm and test it on different graphs to verify its speed and scalability.

How does the algorithm work ?

Data preparation

In the present work, we used Databricks to test and review our implementations on toy graphs.

We chose to use:

- The *Web Google graph* released in 2002 by Google (875K nodes, 5.1M edges). Nodes represent web pages and directed edges represent hyperlinks between them.
- The *Berkeley-Stanford graph* released in 2002 (685K nodes, 7.6M edges) represents pages from berkely.edu and stanford.edu domains and directed edges represent hyperlinks between them.
- The *University of Notre Dame graph* released in 1999 (325K nodes, 1.5M edges) represents pages from nd.edu domain and directed edges represent hyperlinks between them.
- The *Stanford graph* released in 2002 (282K nodes, 2.3M edges) represents pages from stanford.edu domain and directed edges represent hyperlinks between them.

Data was already clean. All we needed to do was removing the headers and make sure nodes are converted to integers.

Example : Data preparation with RDD

```
def clean_data(raw_data):

    # remove the header
    data = raw_data.filter(lambda x : "#" not in x)

    # split data to form (key, value)
    data = data.map(lambda x : x.split("\t"))
                .map(lambda x: (x[0], x[1]))

    return data
```

The CCF Module

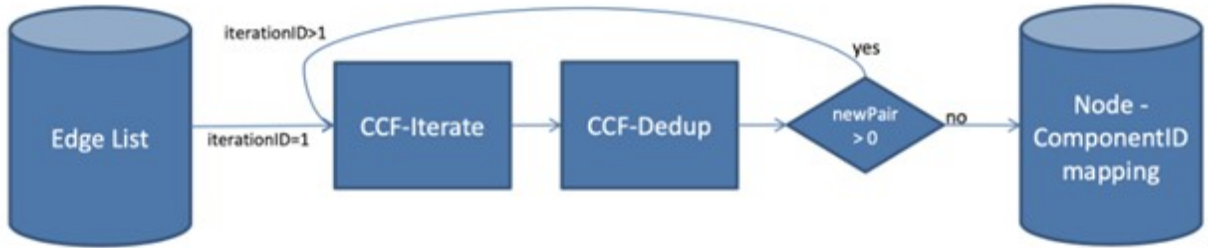


Figure 1: Connected Component Finder(CCF) Module

Inputs are a list of all the edges of the graph. Edges are modeled by a list of (key, value) representing two connected nodes: [node1]–[node2].

Outputs are the mapping from each node in the graph to the smallest node ID in its corresponding connected component (arbitrary choice made in the paper to identify the component).

In between, there is a loop over 2 Map Reduces jobs until all the nodes are assigned to a component ID:

1. CCF-Iterate: link nodes to component ID

CCF-Iterate is responsible for the main iteration of the CCF algorithm. It performs the steps necessary to update node labels and merge related components.

In its map function, it outputs each node in the graph with its own label, as well as each pair of connected nodes in the graph.

In its reduce function, it selects the smallest label from the labels of the connected nodes and propagates this label to all nodes in the connected component.

It also uses a counter to track the number of label updates made during each iteration. This helps determine whether significant changes have been made to the graph structure.

```

CCF-Iterate
map(key, value)
    emit(key, value)
    emit(value, key)

reduce(key, < iterable > values)
    min ← key
    for each (value ∈ values)
        if(value < min)
            min ← value
        valueList.add(value)
    if(min < key)
        emit(key, min)
    for each (value ∈ valueList)
        if(min ≠ value)
            Counter.NewPair.increment(1)
            emit(value, min)

```

Figure 2: CCF-Iterate Algorithm

2. CFF-Dedup: dump copies of outputs of CFF-Iterate

CCF-Dedup is used after the main iteration of CCF-Iterate to eliminate duplicate node labels.

In its map function, it outputs each pair of nodes with a temporary label. In its reduce function, it issues each pair of nodes with a single label, eliminating duplicates.

This consolidates the node label information after the main iteration, ensuring a single representation for each connected component in the graph.

```

CCF-Dedup
map(key, value)
    temp.entity1 ← key
    temp.entity2 ← value
    emit(temp, null)

reduce(key, < iterable > values)
    emit(key.entity1, key.entity2)

```

Figure 3: CCF-Dedup Algorithm

The various implementations

The algorithm described above has been implemented on both Python and Scala, and both using RDDs and Dataframes (see Appendix). The algorithm was then tested in each case on the 4 graphs presented earlier (Google, BerkStan, NotreDame and Stanford). Conclusions on the performance differences between each of these tests are drawn in the following section.

Here is a commented implementation of the algorithm in PySpark with RDDs:

```
# Define the Map Phase of the CCF Iterate
def mapper(rdd):

    # Map each edge to its two vertices and their reverse, preserving both
    mapped = rdd.flatMap(lambda x: ((x[0],x[1]),(x[1],x[0])))

    # Group vertices by key and convert values to lists
    mapped = mapped.groupByKey().mapValues(list)
        .partitionBy(100).cache()

    return mapped.map(lambda x: (int(x[0]), list(map(int, x[1]))))

# Define the Reduce Phase of the CCF Iterate
def reducer(edges):
    key, values = edges
    valueList, emit = [], []

    # Find the minimum value in the list of values
    minim = min(values)
    valueList.extend(values)

    # If the key is less than or equal to the minimum value,
    # return an empty list
    if key <= minim:
        return [], 0

    emit.append((key, minim))

    # Emit key-value pairs where value is the minimum value
    for value in valueList:
        if minim != value:
            emit.append((value, minim))

    # Return emitted pairs and their count
    return emit, len(emit)-1
```

Connected Components Finder Algorithm

```
def find_ccp(rdd):
    start = time.time()
    iterations = 0
    total_new_pairs = 0

    while True:

        # CCFiterate
        mapp = mapper(rdd)
        reduce = mapp.map(lambda x: reducer(x))
        # Count new pairs generated in this iteration
        new_pairs = reduce.map(lambda x: x[1])
            .reduce(lambda x, y: x + y)
        total_new_pairs += new_pairs
        result_reducer = reduce.flatMap(lambda x: x[0])

        # CCFdedup
        rdd = result_reducer.distinct()

        # Print iteration number and number of new pairs generated
        print(f"Iteration-{iterations}")
        print(f"Number-of-new-pairs:-{new_pairs}")

        # Break the loop if no new pairs were generated
        if new_pairs == 0:
            break

        iterations += 1

    # Calculate and print the number of connected components and the
    # size of the largest component
    time_spent = time.time() - start
    number_of_components = rdd.map(lambda x: x[1]).distinct().count()
    largest_component_size = rdd.map(lambda x: (x[1], 1))
        .reduceByKey(lambda x, y: x + y).map(lambda x: x[1]).max()

    print(f"\nNumber-of-components:-{number_of_components}")
    print(f"The-largest-component-has-{largest_component_size}-elements")
    print(f"Total-elapsed-time:-{time_spent}-s")
```

Scalability: how did it work on different graphs ?

	# edges	# nodes	RDD Python	RDD Scala	DF Python	DF Scala
Hand-crafted (test)	5	8	44 sec	9 sec	42 sec	30 sec
NotreDame	1 497 134	325 729	275 sec	262 sec	421 sec	384 sec
Stanford	2 312 497	281 903	584 sec	568 sec	956 sec	878 sec
Google	5 105 039	875 713	721 sec	619 sec	1 339 sec	1 063 sec
BerkStan	7 600 595	685 230	879 sec	1 310 sec	1 756 sec	1 212 sec

Figure 4: Execution time of the CCF algorithm on Databricks

Above we can see the execution times of our algorithms in several situations. These executions were carried out on Databricks Community Edition Standard Clusters, equipped with 2 Cores.

We can see that execution times were generally lower when the algorithm was coded in Scala, except in the case of the largest graph (BerkStan in RDD), where we can see that scaling was very poor.

We can also see that execution times are much longer when using DataFrames than when using RDDs. This can be explained by the additional processing costs associated with DataFrames. Indeed, DataFrames introduce an abstraction overlay which facilitates the processing of structured data, but which can also entail additional transformation and optimization operations. All this, and perhaps something else, can slow down performance compared with the raw use of RDDs. As a result, although DataFrames offer a more user-friendly syntax and additional functionality, their use can lead to a significant increase in execution times for complex operations, in our situation on Databricks.

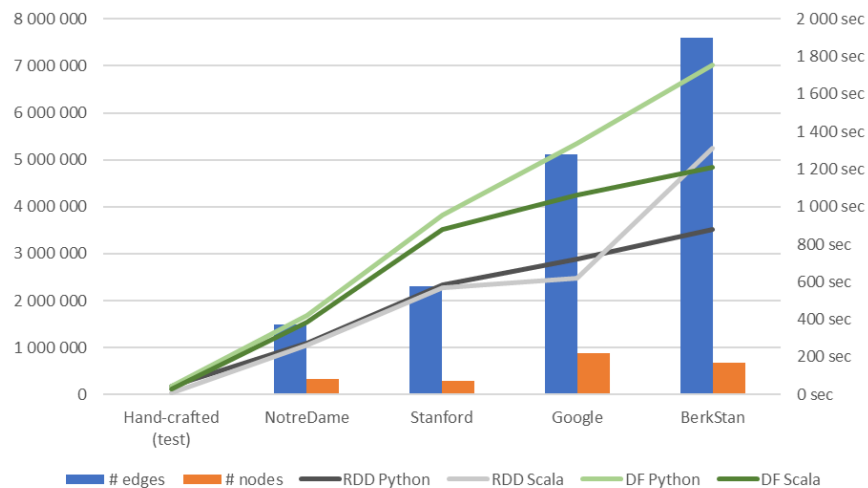


Figure 5: Execution time with regards to number of nodes and edges

As we can see from this graph, which compares the execution time of the algorithm with RDDs in Scala (green) and Python (yellow), it seems that the increase in execution time is correlated with the increase in the number of edges, rather than the number of nodes.

We can also see, from the execution time achieved for the BerkStan graph, a better scalability for Scala code.

Conclusion

In conclusion, we find that the algorithm maintains acceptable execution times even with large graphs, indicating good scalability in environments where graphs can be quite large. However, we also observe significant signs of slowdown in some cases, suggesting potential scalability limitations in certain scenarios (such as BerkStan). Although our algorithm demonstrates satisfactory scalability in many use cases, there are still areas where improvements are needed to guarantee optimal performance in environments with increasingly large graphs.

References

1. Karden, H., Agrawal, S., Wang, X., & Sun, A. (2014). CCF: Fast and scalable connected component computation in MapReduce. *IEEE Transactions on Knowledge and Data Engineering*, 26(10), 2455-2467.

Appendix

CCF with PySpark and RDDs for the Google graph

```
# Databricks notebook source
from pyspark import SparkContext, SparkConf
import time

## Data preparation with RDD
def clean_data(raw_data):
    # remove the header
    data = raw_data.filter(lambda x : "#" not in x)
    # split data to form (key, value)
    data = data.map(lambda x : x.split("\t")).map(lambda x: (x[0], x[1]))
    return data

def mapper(rdd):
    mapped = rdd.flatMap(lambda x: ((x[0],x[1]),(x[1],x[0])))
    mapped = mapped.groupByKey().mapValues(list).partitionBy(100).cache()
    return mapped.map(lambda x: (int(x[0]), list(map(int, x[1]))))

def reducer(edges):
    key, values = edges
    valueList, emit = [], []
    minim = min(values)
    valueList.extend(values)

    if key <= minim:
        return [], 0

    emit.append((key, minim))
    for value in valueList:
        if minim != value:
            emit.append((value, minim))

    return emit, len(emit)-1

def find_ccp(rdd):

    start = time.time()
    iterations = 0
    total_new_pairs = 0

    while True:
        # CCFiterate
        mapp = mapper(rdd)
        reduce = mapp.map(lambda x: reducer(x))
```

```

    new_pairs = reduce.map(lambda x: x[1]).reduce(lambda x, y: x + y)
    total_new_pairs += new_pairs
    result_reducer = reduce.flatMap(lambda x: x[0])

    # CCFdedup
    rdd = result_reducer.distinct()

    print(f"iteration-{iterations}")
    print(f"#new-pairs:-{new_pairs}")

    if new_pairs == 0:
        break

    iterations += 1

time_spent = time.time() - start
number_of_components = rdd.map(lambda x: x[1]).distinct().count()
largest_component_size = rdd.map(lambda x: (x[1], 1))
    .reduceByKey(lambda x, y: x + y).map(lambda x: x[1]).max()

print(f"\n#components:-{number_of_components}")
print(f"The-largest-component-has-{largest_component_size}-elements")
print(f"Total-elapsed-time:-{time_spent}-s")

web_google_path = "/FileStore/shared-uploads/benjamin.cerf
-----@polytechnique.edu/web-Google.txt"
web_google = spark.read.text(web_google_path)

# Convert DataFrame to RDD
google_rdd = web_google.rdd.map(lambda row: row.value)

# Now apply our clean_data function
clean_google_rdd = clean_data(google_rdd)

find_ccp(clean_google_rdd)

```

CCF with Scala and RDDs for the Google graph

```
// Databricks notebook source
import org.apache.spark.rdd.RDD
import scala.collection.mutable.ListBuffer

def cleanData(rawData: RDD[String]): RDD[(Int, Int)] = {
  rawData
    .filter(!_.startsWith("#")) // Remove lines starting with "#"
    .map(_.split("\\s+")) // Split each line on whitespace
    .filter(_.length == 2) // Ensure each line has exactly two elements
    .map(parts => (parts(0).toInt, parts(1).toInt)) // Convert to (Int, Int) pairs
}

def mapper(rdd: RDD[(Int, Int)]): RDD[(Int, List[Int])] = {
  val mapped = rdd.flatMap(x => List((x._1, x._2), (x._2, x._1)))
  mapped.groupByKey().mapValues(_.toList)
  .partitionBy(new org.apache.spark.HashPartitioner(100)).cache()
  .map(x => (x._1.toInt, x._2.map(_.toInt).toList))
}

def reducer(edges: (Int, Iterable[Int])): (List[(Int, Int)], Int) = {
  val (key, values) = edges
  val valueList = values.toList
  val minim = valueList.min
  val emit = ListBuffer[(Int, Int)]()

  if (key <= minim) {
    return (List(), 0)
  }

  emit.append((key, minim))
  valueList.foreach { value =>
    if (minim != value) {
      emit.append((value, minim))
    }
  }

  (emit.toList, emit.size - 1)
}

def find_ccp(rdd: RDD[(Int, Int)]): Unit = {
  val start = System.currentTimeMillis()
  var iterations = 0
  var total_new_pairs = 0
  var newRDD = rdd
  var continue = true

```

```

while (continue) {
  // CCFiterate
  val mapp = mapper(newRDD)
  val reduce = mapp.map(x => reducer(x))
  val new_pairs = reduce.map(x => x._2).reduce(_ + _)
  total_new_pairs += new_pairs
  val result_reducer = reduce.flatMap(x => x._1)

  // CCFdedup
  newRDD = result_reducer.distinct()

  println(s"Iteration-$iterations")
  println(s"#new-pairs:-$new_pairs")

  if (new_pairs == 0) {
    continue = false
  } else {
    iterations += 1
  }
}

val time_spent = (System.currentTimeMillis() - start) / 1000.0
val number_of_components = newRDD.map(x => x._2).distinct().count()
val largest_component_size = newRDD.map(x => (x._2, 1))
  .reduceByKey(_ + _).map(_._2).max()

println(s"\n#components:-$number_of_components")
println(s"The-largest-component-has-$largest_component_size-elements")
println(s"Total-elapsed-time:-$time_spent-s")
}

// Define the file path and read the text files
val web_google_path = "/FileStore/shared_uploads
---/benjamin.cerf@polytechnique.edu/web-Google.txt"
val web_google = spark.read.text(web_google_path)

// MAGIC Web Google

// Convert DataFrame to RDD[String]
val web_google_rdd = web_google.rdd.map(row => row.getString(0))

// Apply the cleanData function
val cleaned_google_RDD = cleanData(web_google_rdd)

find_ccp(cleaned_google_RDD)

```

CCF with PySpark and Dataframes for the Google graph

```

# Databricks notebook source
import pyspark
from pyspark.sql import Window
from pyspark.sql.functions import *
from pyspark import SparkContext as sc
from pyspark.sql import SparkSession
import time

def g_preprocessing(graph):
    def filter_lines(tup):
        # Filter lines wrt index
        return tup[1] > 3

    def split_values(line):
        # map each line in list of values
        return line.split('\t')

    filtered_graph = graph.zipWithIndex().filter(filter_lines)
    mapped_graph = filtered_graph.map(split_values)

    return mapped_graph

def mapper(df):
    new_row = df.select("val", "key")
    mapped_df = df.union(new_row)
    return mapped_df

def reducer(mapped_df):
    window = Window.orderBy("key", "val").partitionBy("key")
    df_with_min = mapped_df.withColumn("min_val", min("val").over(window))

    new_key_expr = expr(
        "CASE-WHEN min_val > key THEN NULL ELSE CASE-WHEN
        min_val = val THEN key ELSE val END-END"
    )
    new_val_expr = expr(
        "CASE-WHEN min_val > key THEN NULL ELSE min_val END"
    )

    reduced_df = (df_with_min
        .withColumn("new_key", new_key_expr)
        .withColumn("new_val", new_val_expr)
        .na.drop()
        .select(col("new_key").alias("key"), col("new_val").alias("val")))

```

```

        .sort("val", "key"))

    distinct_reduced_df = reduced_df.distinct()

    return distinct_reduced_df, df_with_min

def ccf_algo(df):
    start = time.time()
    count = 1
    iterations = 0

    while count!= 0:
        iterations += 1
        mapped_df = mapper(df)
        mapped_df.cache()
        df.unpersist()
        df, df_counter = reducer(mapped_df)
        df.cache()
        mapped_df.unpersist()

        df_counter = df_counter.withColumn("counter_col", expr(
            "CASE-WHEN min_val > key THEN 0 ELSE CASE-WHEN min_val
            -----= val THEN 0 ELSE 1 END-END"
        ))

        count = df_counter.select(sum("counter_col")).collect()[0][0]

    time_spent = time.time() - start
    number_of_components = df.select("val").distinct().count()

    print(f"\n#components:-{number_of_components}")
    print(f"Total elapsed time:-{time_spent}-s")

graph = spark.sparkContext.textFile("/FileStore/shared_uploads/
benjamin.cerf@polytechnique.edu/web-Google.txt")

graph = g_preprocessing(graph)
graph.collect()

df_google = spark.createDataFrame(graph, ["key", "val"])
df_google.show()

ccf_algo(df_google)

```

CCF with Scala and Dataframes for the Google graph

```
// Databricks notebook source
import org.apache.spark.sql.Session
import org.apache.spark.sql.expressions.Window
import org.apache.spark.sql.functions._
import spark.implicits._

def g_preprocessing(graph: org.apache.spark.rdd.RDD[String]):
  org.apache.spark.rdd.RDD[(String, String)] = {
    // Filter lines wrt index
    def filter_lines(tup: (String, Long)): Boolean = {
      tup._2 > 3
    }

    // Split values of each line
    def split_values(line: String): (String, String) = {
      val Array(key, value) = line.split('\t')
      (key, value)
    }

    val filtered_graph = graph.zipWithIndex().filter(filter_lines)
      .map(_._1)
    val mapped_graph = filtered_graph.map(split_values)

    mapped_graph
  }

def mapper(df: org.apache.spark.sql.DataFrame): org.apache.spark.sql
  .DataFrame = {
  val new_row = df.select($"val", $"key")
  val mapped_df = df.union(new_row)
  mapped_df
}

def reducer(mapped_df: org.apache.spark.sql.DataFrame):
  (org.apache.spark.sql.DataFrame, org.apache.spark.sql.DataFrame) = {
  val window = Window.orderBy("key", "val").partitionBy("key")
  val df_with_min = mapped_df.withColumn("min_val", min("val")
    .over(window))

  val new_key_expr = expr(
    "CASE-WHEN min_val > key THEN NULL ELSE CASE-WHEN min_val
    -> val THEN key ELSE val END-END"
  )
  val new_val_expr = expr(
    "CASE-WHEN min_val
    -> key THEN NULL ELSE min_val END"
  )
}
```

```

    val reduced_df = df_with_min
      .withColumn("new_key", new_key_expr)
      .withColumn("new_val", new_val_expr)
      .na.drop()
      .select(col("new_key").alias("key"), col("new_val").alias("val"))
      .sort("val", "key")

    val distinct_reduced_df = reduced_df.distinct()
    (distinct_reduced_df, df_with_min)
  }

def ccf_algo(df: org.apache.spark.sql.DataFrame): org.apache.spark.sql
  .DataFrame = {
  val start = System.currentTimeMillis()
  val count = 1
  val iterations = 0
  val newDF = df

  while (count != 0) {
    iterations += 1
    val mapped_df = mapper(newDF)
    newDF.unpersist()
    val (reduced_df, df_counter) = reducer(mapped_df)
    df_counter.createOrReplaceTempView("df_counter")
    val countDF = spark.sql("SELECT-SUM(counter_col)-FROM-(SELECT-CASE
---WHEN min_val > key THEN 0 ELSE CASE WHEN min_val = val THEN 0 ELSE 1
---END-END-AS counter_col FROM df_counter)")
    count = countDF.collect()(0)(0).toString.toInt
    newDF = reduced_df.cache()
    mapped_df.unpersist()
  }

  val time_spent = (System.currentTimeMillis() - start) / 1000.0
  println(s"Total elapsed time: $time_spent s")
}

// Read text file into RDD
val graphRDD = spark.sparkContext.textFile("/FileStore/shared_uploads
/benjamin.cerf@polytechnique.edu/web_Google.txt")

// Preprocess the graphRDD
val preprocessedGraph = g_preprocessing(graphRDD)

// Convert RDD to DataFrame
val dfGoogle = preprocessedGraph.toDF("key", "val")

ccf_algo(dfGoogle)

```